

4-对抗搜索 Minimax

对抗搜索 Adversarial Search

之前提到的有信息、无信息搜索都是非对抗性的，只有一个智能体agent，不会有他人干预

类似local search[3-CSP LocalSearch](#)的评估函数，游戏中有个效用函数Utility(state,player) -> value，用于评估游戏得分

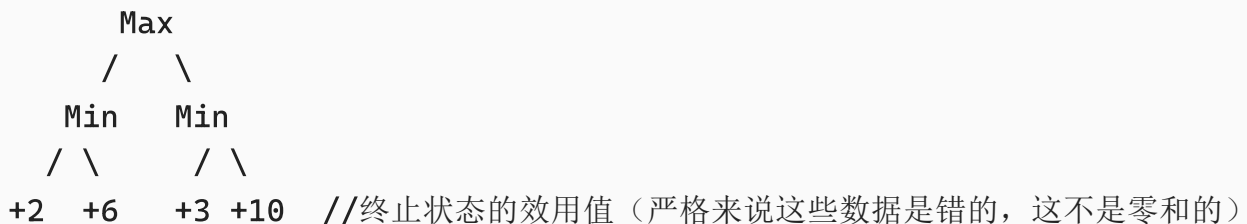
这节课提到的游戏暂时是

- 确定性的，也就是玩家可以知道所有的游戏状态（没有随机生成，没有看不到他人手牌）
- 两个玩家轮流进行
- 零和的（player1的utility为a，那么player2就是-a）

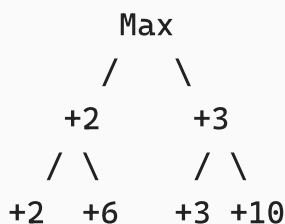
极小化极大算法 Minimax

根据上述对游戏的假设，有两方玩家，一方称为max玩家，目标是最大化自己的收益；一方是min玩家，目标是最小化 Max 的收益（等同于最大化自己的收益，因为这是零和博弈）
两个玩家的回合在游戏树中呈现为两个节点，Max 节点会选择子节点中的最大值；Min 节点会选择子节点中的最小值。

默认假设max玩家先开始回合



对于两个Min节点，会挑选最小的子节点



因此Max节点选择+3，最后max玩家沿着+3的路径采取行动

```
def minimax(node, depth, is_maximizing):
    # 终止条件：达到最大深度（如果游戏回合无限或陷入循环）或游戏结束
    if depth == 0 or node.is_terminal():
        return evaluate(node)
```

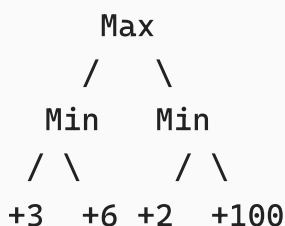
```

if is_maximizing:
    # Max 玩家：选择最大值
    best_value = -float('inf')
    for child in node.children():
        value = minimax(child, depth-1, False) # 递归
        best_value = max(best_value, value)
    return best_value
else:
    # Min 玩家：选择最小值
    best_value = float('inf')
    for child in node.children():
        value = minimax(child, depth-1, True) # 递归
        best_value = min(best_value, value)
    return best_value

```

递归地遍历整个博弈树，直到游戏结束（评估状态的效用值），然后回溯来选择最优路径

- 搜索过程和复杂度都类似DFS[1-搜索算法](#)，时间复杂度 $O(b^m)$ ，因而对于国际象棋，基本是不可能得到结果的
- 假设两个玩家都是optimal的，但是如果对手有一定犯错的概率，就无法计算出可能的更好结果，比如：



右侧的min节点，如果玩家有一半的概率犯错，max就可以得到+100，就算没犯错也是+2，没有比+3差太多

但是minimax算法总会让max选择+3的这条路径

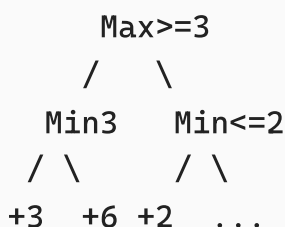
优化：Alpha-Beta Pruning剪枝

不需要访问整个搜索树也可以得到答案

alpha = 到目前为止，路径上发现的max的任一节点中最佳（即最大值）选择的值。

beta = 到目前为止，路径上发现的min的任一节点中最佳（即最小值）选择的值。

例如以下情形：

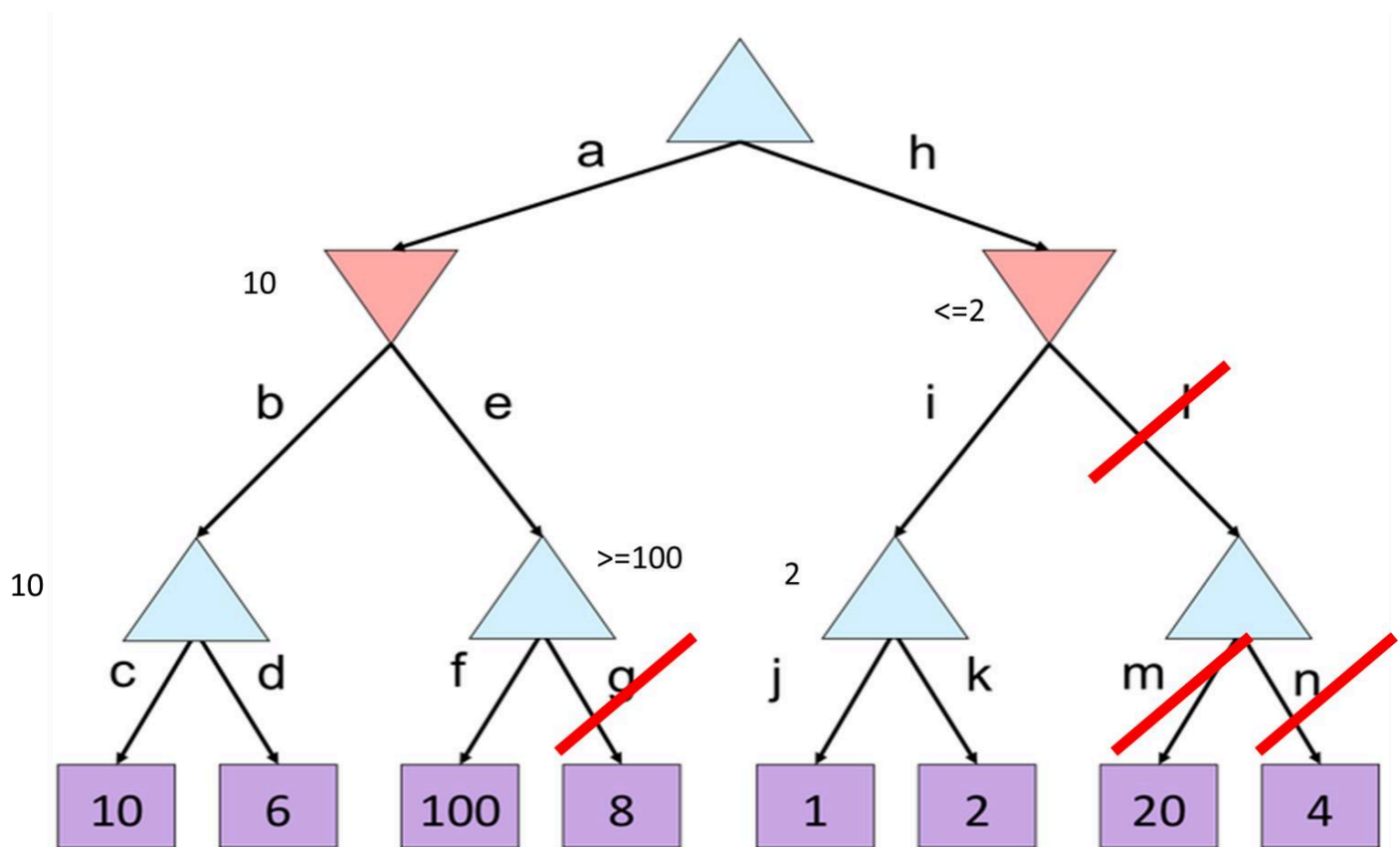


搜索左侧树后，知道顶端的Max至少保证是+3，继续访问新节点也只增不减

右侧的Min访问子节点，发现是+2，那么这个Min将确保选择 $\leq +2$ 的值，后续访问只减不增

然而对于顶端的Max来说，既然自己已经有了+3的选项，而这个Min节点最大也是+2，那就不用探索剩下部分了（剪枝，...表达）

又如：（上三角是max，下三角是min）



g l 分支就被除去，不用访问了（图中m n被划去，实际上根本不会访问到m n，因为l被除去）

注意alpha beta是对每个节点独立的，虽然会反向传播到上层。而且alpha beta不是节点的最终效用值

```
def alphabeta(node, alpha, beta, depth, maximizing):
    if depth == 0 or node.is_terminal():
        return evaluate(node)

    if maximizing:
        v = float('-inf')
        for child in node.children:
            v = max(v, alphabeta(child, alpha, beta, depth - 1, False))
            if v >= beta: # beta 剪枝: 当前最大值已超出 Min 能容忍的范围
                return v
            alpha = max(alpha, v) # 更新 alpha
        return v
    else:
        v = float('inf')
        for child in node.children:
            v = min(v, alphabeta(child, alpha, beta, depth - 1, True))
```

```

        if v <= alpha: # alpha 剪枝: 当前最小值已低于 Max 能保证的范围
            return v
        beta = min(beta, v) # 更新 beta
    return v

```

经过这样优化以及子节点最优的选择顺序，时间复杂度可以降到 $O(b^{(m/2)})$

优化：深度受限搜索 Depth-limited Search

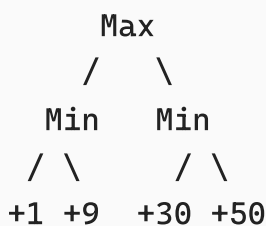
alpha-beta剪枝虽然优化成了平方根，但是复杂度还是**指数级**增长的，如果搜索树还是很大，递归会过深

从而，设置该次搜索的最大深度上限，如果达到上限，就算最后访问的不是终止节点也要停止

但是utility效用函数只能评估终止状态，所以需要有一个评估函数Evaluation Function，对非终止状态打分，然后进行minimax的回溯

此外这个评估函数也可以帮助alpha-beta剪枝，因为

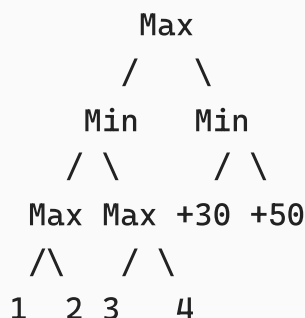
- 剪枝效率受节点的**扩展顺序**影响很大，如果生成后继节点的时候可以先生成有希望的节点，后续更容易触发剪枝



对于max来说，如果生成后继节点的顺序是先出右min节点，就可以得到30的alpha下界，左节点访问一个+1就可以直接剪枝了

可以用优先队列，用评估函数给后继节点打分来实现

- 如果评估函数能给节点一个**正确的界限**，不需要访问到叶子节点就可以剪枝



假设先扩展右Min节点，得到30的alpha，然后评估左Min节点，假设评估出一个上界5，就直接剪枝，不需要一路访问到终止节点

当然前提是评估函数能给正确的界限

可以结合迭代加深的方法

期望最大搜索

下节课